PYTHON PROGRAMMING
PRACTICAL RECORD

AIM ● PROCEDURE ● FLOWCHART ● PROGRAM ● OUTPUT

Combined record of all 10 uploaded Python programs

Flowchart Diagram Format

OVAL / TERMINATOR	**Start or Stop**
PARALLELOGRAM	Input or Output
RECTANGLE	Processing / Calculation
DIAMOND	Decision / Condition
ARROW (↓)	Direction of flow

Each flowchart is shown as a vertical diagram using rectangular boxes connected by arrows.

Experiment 1: Employee Payroll and Deduction Analysis

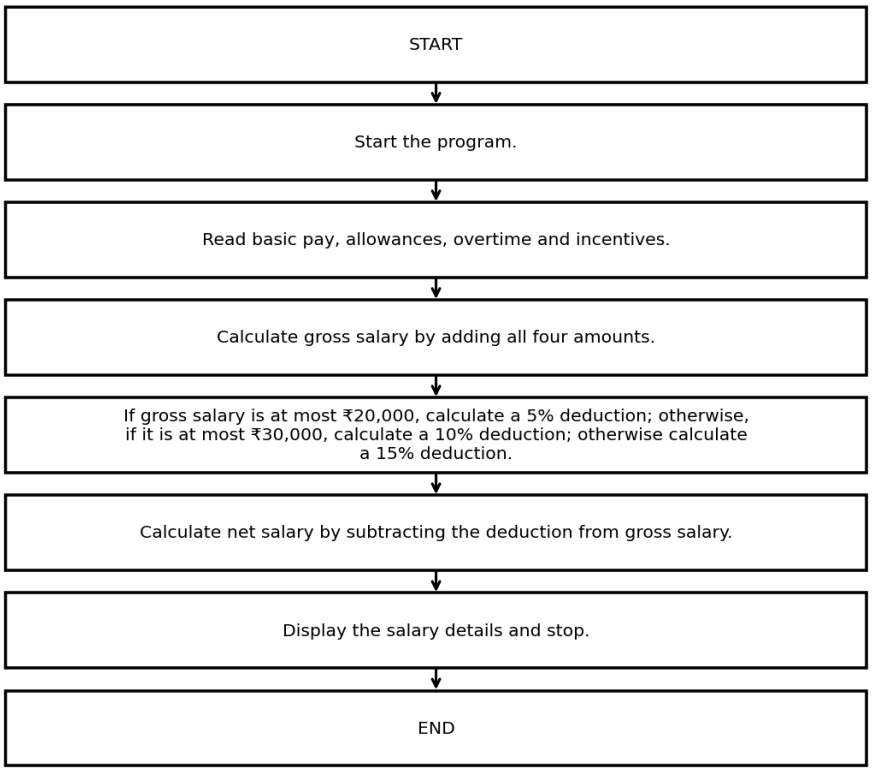
Aim

To calculate an employee's gross salary from basic pay, allowances, overtime and incentives, apply a deduction rate based on the gross salary, and calculate the net salary.

Procedure

1. Start the program.
2. Read basic pay, allowances, overtime and incentives.
3. Calculate gross salary by adding all four amounts.
4. If gross salary is at most ₹20,000, calculate a 5% deduction; otherwise, if it is at most ₹30,000, calculate a 10% deduction; otherwise calculate a 15% deduction.
5. Calculate net salary by subtracting the deduction from gross salary.
6. Display the salary details and stop.

Flowchart



Program

```
basic=int(input("enter a number:"))
allowances=int(input("enter a number:"))
overtime=int(input("enter a number:"))
incentives=int(input("enter anumber:"))
gross=basic+allowances+overtime+incentives
if gross<=20000:
   deduction=gross*0.05
elif gross<=30000:
   deduction=gross*0.10
else:
   deduction=gross*0.15
```

```
net=gross-deduction
print("Basic=",basic)
print("Allowances=",allowances)
print("Overtime=",overtime)
print("Incentives=",incentives)
print("Deduction=",deduction)
print("Gross=",gross)
```

Output

Sample input:
Basic = 15000
Allowances = 3000
Overtime = 1000
Incentives = 1000

Output:
Basic= 15000
Allowances= 3000
Overtime= 1000
Incentives= 1000
Deduction= 1000.0
Gross= 20000

Note: The program calculates net salary but does not print it.

Experiment 2: Passenger Count and Bus Capacity Monitoring

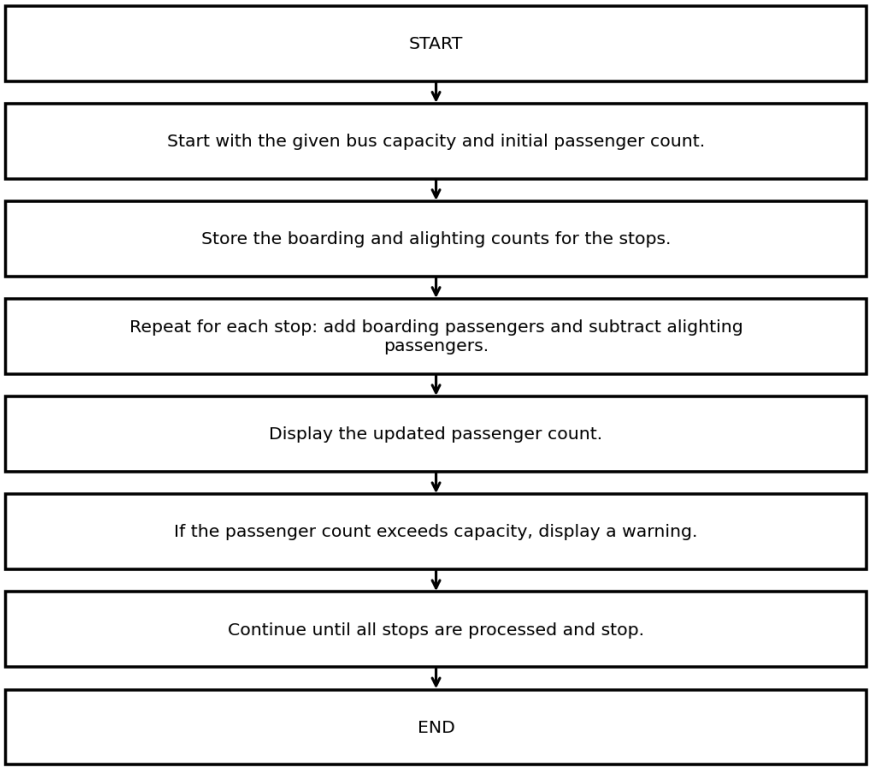
Aim

To update the number of passengers after boarding and alighting at each stop and check whether the bus capacity is exceeded.

Procedure

7. Start with the given bus capacity and initial passenger count.
8. Store the boarding and alighting counts for the stops.
9. Repeat for each stop: add boarding passengers and subtract alighting passengers.
10. Display the updated passenger count.
11. If the passenger count exceeds capacity, display a warning.
12. Continue until all stops are processed and stop.

Flowchart



Program

```
capacity = 40
passengers = 10
boarding = [5, 10, 8]
alighting = [2, 5, 3]

for i in range(3):
    passengers = passengers + boarding[i] - alighting[i]
    print("Stop", i + 1, "Passengers:", passengers)
    if passengers > capacity:
        print("Capacity exceeded!")
```

Output

Stop 1 Passengers: 13
Stop 2 Passengers: 18
Stop 3 Passengers: 23

Experiment 3: Bank Transaction and Suspicious Withdrawal Detection

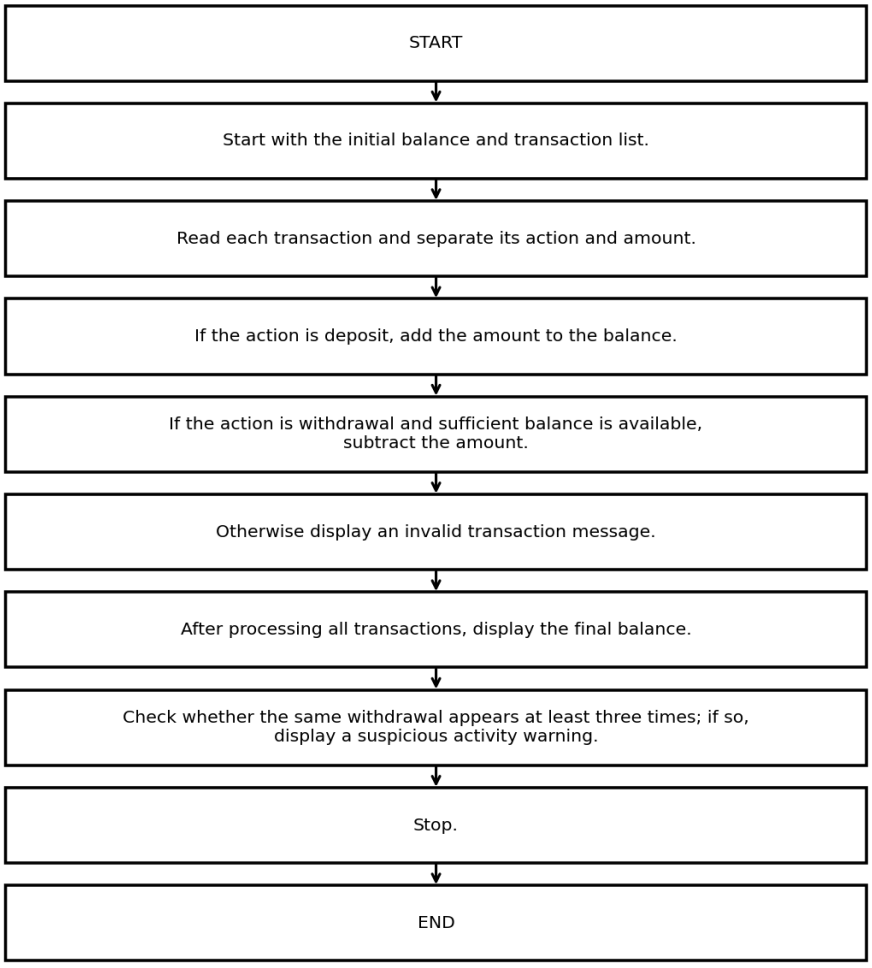
Aim

To process a sequence of deposits and withdrawals, update the account balance and identify repeated withdrawals.

Procedure

13. Start with the initial balance and transaction list.
14. Read each transaction and separate its action and amount.
15. If the action is deposit, add the amount to the balance.
16. If the action is withdrawal and sufficient balance is available, subtract the amount.
17. Otherwise display an invalid transaction message.
18. After processing all transactions, display the final balance.
19. Check whether the same withdrawal appears at least three times; if so, display a suspicious activity warning.
20. Stop.

Flowchart



Program

```
balance = 1000
transactions = ["deposit 500", "withdraw 200", "withdraw 200", "withdraw 200"]

for t in transactions:
    action, amount = t.split()
```

```
    amount = int(amount)

    if action == "deposit":
        balance += amount
    elif action == "withdraw" and amount <= balance:
        balance -= amount
    else:
        print("Invalid transaction!")

print("Final Balance: ₹", balance)

if transactions.count("withdraw 200") >= 3:
    print("Suspicious repeated withdrawals!")
```

Output

Final Balance: ₹ 900
Suspicious repeated withdrawals!

Experiment 4: Classroom Seating Arrangement

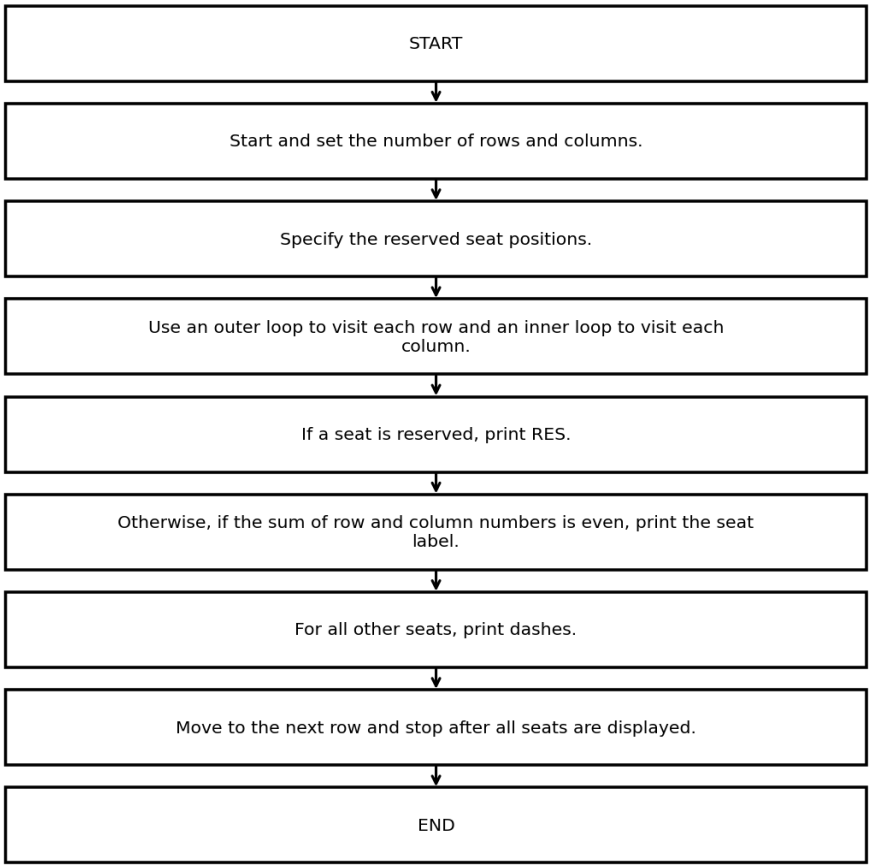
Aim

To generate a 3-row by 4-column seating layout, mark reserved seats and display seat labels according to a condition.

Procedure

21. Start and set the number of rows and columns.
22. Specify the reserved seat positions.
23. Use an outer loop to visit each row and an inner loop to visit each column.
24. If a seat is reserved, print RES.
25. Otherwise, if the sum of row and column numbers is even, print the seat label.
26. For all other seats, print dashes.
27. Move to the next row and stop after all seats are displayed.

Flowchart



Program

```
rows = 3
cols = 4
reserved = [(1, 2), (3, 3)]

for i in range(1, rows + 1):
    for j in range(1, cols + 1):
        if (i, j) in reserved:
            print("RES", end="\t")
        elif (i + j) % 2 == 0:
```

```
            print(f"R{i}C{j}", end="\t")
        else:
            print("---", end="\t")
    print()
```

Output

```
R1C1    RES     R1C3    ---
---     R2C2    ---     R2C4
R3C1    ---     RES     ---
```

Experiment 5: Production Cost and Profit Target Analysis

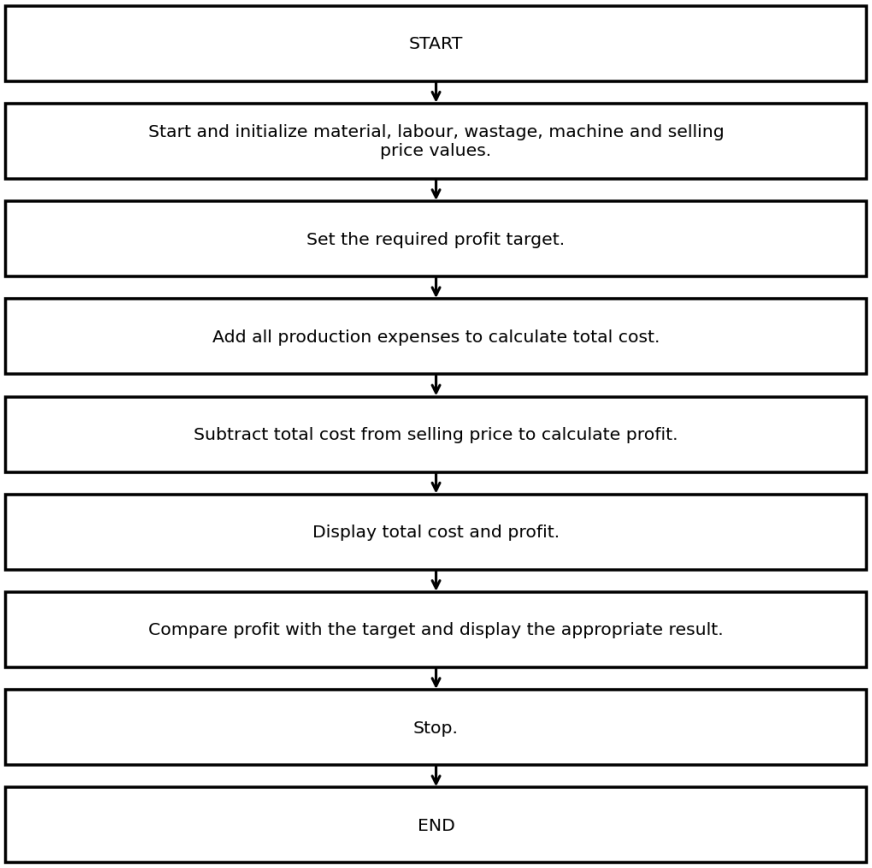
Aim

To calculate the total production cost and profit, then determine whether the profit target has been achieved.

Procedure

28. Start and initialize material, labour, wastage, machine and selling price values.
29. Set the required profit target.
30. Add all production expenses to calculate total cost.
31. Subtract total cost from selling price to calculate profit.
32. Display total cost and profit.
33. Compare profit with the target and display the appropriate result.
34. Stop.

Flowchart



Program

```
material = 5000
labour = 2000
wastage = 500
machine = 1000
selling_price = 11000
target = 2000

cost = material + labour + wastage + machine
profit = selling_price - cost
```

```
print("Total Cost:", cost)
print("Profit:", profit)

if profit >= target:
    print("Profit Target Achieved")
else:
    print("Profit Target Not Achieved")
```

Output

Total Cost: 8500
Profit: 2500
Profit Target Achieved

Experiment 6: Smart Parking Fee Calculation (Car/Bike – Version 1)

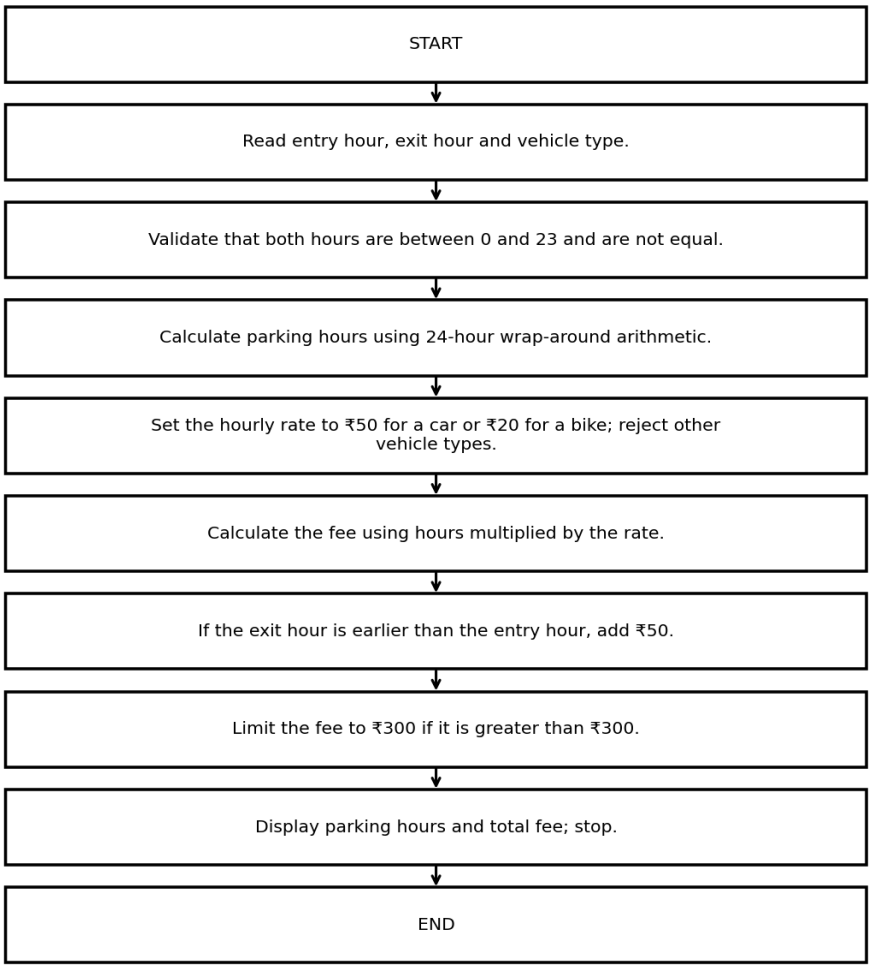
Aim

To calculate parking duration and fee based on entry time, exit time and vehicle type, including an overnight charge and a maximum fee limit.

Procedure

35. Read entry hour, exit hour and vehicle type.
36. Validate that both hours are between 0 and 23 and are not equal.
37. Calculate parking hours using 24-hour wrap-around arithmetic.
38. Set the hourly rate to ₹50 for a car or ₹20 for a bike; reject other vehicle types.
39. Calculate the fee using hours multiplied by the rate.
40. If the exit hour is earlier than the entry hour, add ₹50.
41. Limit the fee to ₹300 if it is greater than ₹300.
42. Display parking hours and total fee; stop.

Flowchart



Program

```
entry = int(input("Enter entry hour (0-23): "))
exit = int(input("Enter exit hour (0-23): "))
vehicle = input("Enter vehicle (car/bike): ").lower()
```

```
if entry < 0 or entry > 23 or exit < 0 or exit > 23 or entry == exit:
    print("Invalid time")
else:
    hours = (exit - entry) % 24

    if vehicle == "car":
        rate = 50
    elif vehicle == "bike":
        rate = 20
    else:
        rate = 0

    if rate == 0:
        print("Invalid vehicle")
    else:
        fee = hours * rate

        if exit < entry:
            fee = fee + 50

        if fee > 300:
            fee = 300

        print("Parking hours:", hours)
        print("Total parking fee: ", fee)
```

Output

Sample input:
Enter entry hour (0-23): 9
Enter exit hour (0-23): 12
Enter vehicle (car/bike): car

Output:
Parking hours: 3
Total parking fee: 150

Experiment 7: Campus Event Financial Planning

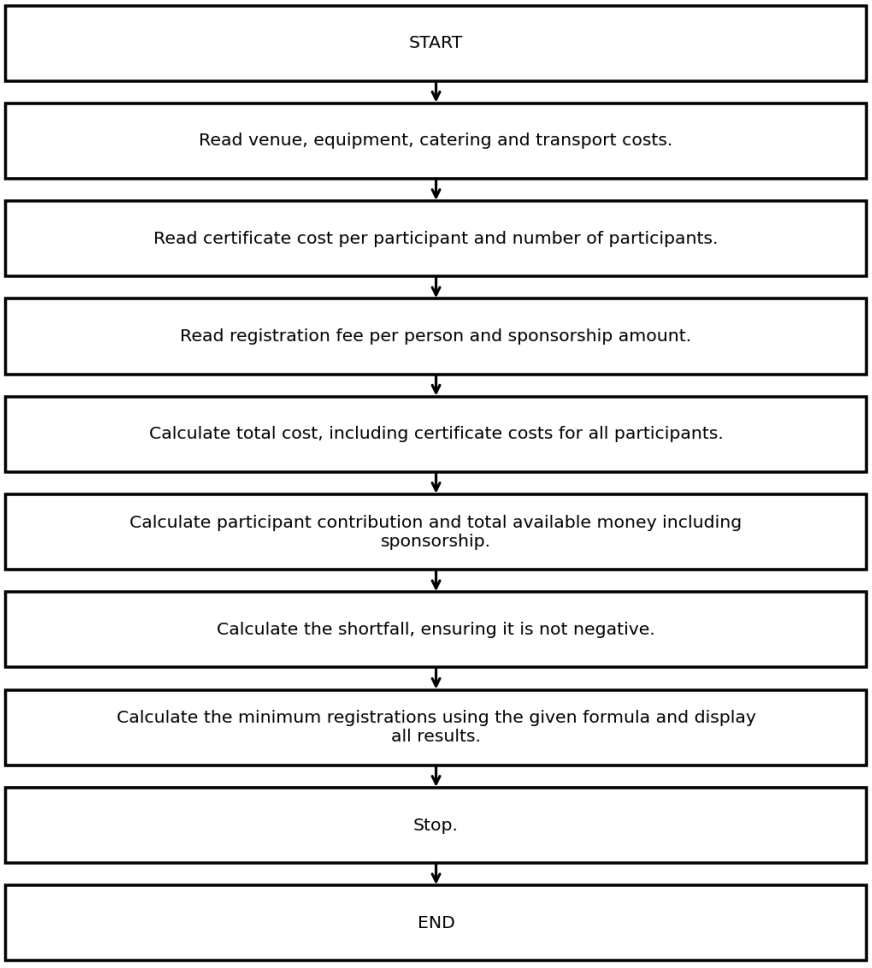
Aim

To calculate the total cost of a campus event, participant contribution, funding shortfall and minimum registrations required.

Procedure

43. Read venue, equipment, catering and transport costs.
44. Read certificate cost per participant and number of participants.
45. Read registration fee per person and sponsorship amount.
46. Calculate total cost, including certificate costs for all participants.
47. Calculate participant contribution and total available money including sponsorship.
48. Calculate the shortfall, ensuring it is not negative.
49. Calculate the minimum registrations using the given formula and display all results.
50. Stop.

Flowchart



Program

```
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost: "))
transport = float(input("Enter transport cost: "))
certificate = float(input("Enter certificate cost per person: "))
```

```
n = int(input("Enter number of participants: "))
fee = float(input("Enter registration fee per person: "))
sponsor = float(input("Enter sponsorship amount: "))

total = venue + equipment + catering + transport + certificate * n
money = n * fee + sponsor
print("Total Cost:", total)
print("Participant Contribution:", n * fee)
print("Shortfall:", max(0, total - money))
print("Minimum Registrations:", int((venue + equipment + catering + transport - sponsor) / (fee - certificate)) + 1)
```

Output

Sample input:
Venue = 10000
Equipment = 5000
Catering = 8000
Transport = 2000
Certificate per person = 100
Participants = 100
Registration fee per person = 300
Sponsorship = 5000

Output:
Total Cost: 35000.0
Participant Contribution: 30000.0
Shortfall: 0
Minimum Registrations: 101

Experiment 8: Attendance Recovery Computation

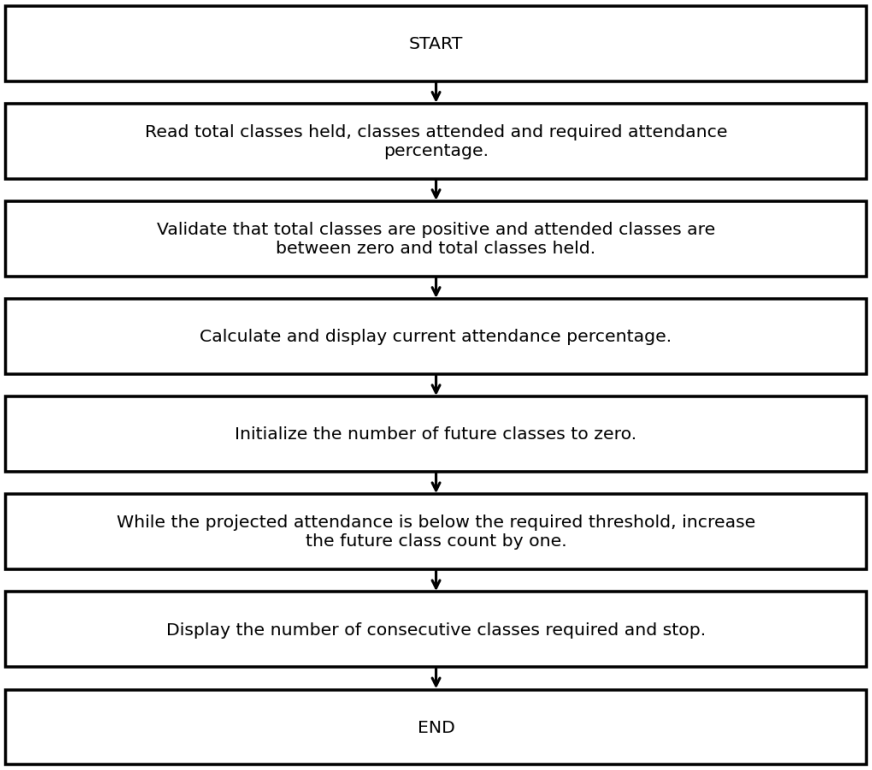
Aim

To calculate current attendance percentage and determine the number of consecutive future classes required to reach the required attendance percentage.

Procedure

51. Read total classes held, classes attended and required attendance percentage.
52. Validate that total classes are positive and attended classes are between zero and total classes held.
53. Calculate and display current attendance percentage.
54. Initialize the number of future classes to zero.
55. While the projected attendance is below the required threshold, increase the future class count by one.
56. Display the number of consecutive classes required and stop.

Flowchart



Program

```
held = int(input("Total classes held: "))
attended = int(input("Classes attended: "))
threshold = float(input("Required attendance %: "))

if held <= 0 or attended < 0 or attended > held:
    print("Invalid attendance details")
else:
    percent = attended / held * 100
    print("Current Attendance:", round(percent, 2), "%")
    future = 0
    while (attended + future) / (held + future) * 100 < threshold:
```

```
        future += 1
    print("Consecutive classes required:", future)
```

Output

Sample input:
Total classes held: 60
Classes attended: 30
Required attendance %: 75

Output:
Current Attendance: 50.0 %
Consecutive classes required: 60

Experiment 9: Smart Parking Fee Management (Car/Bike – Version 2)

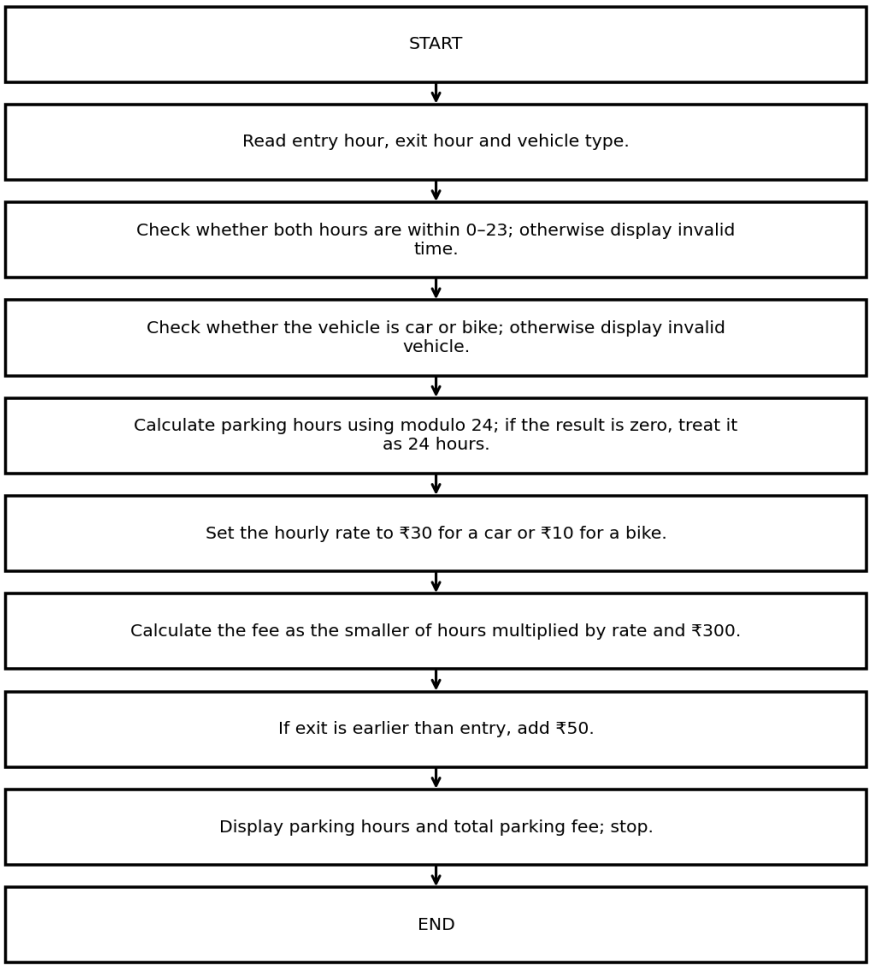
Aim

To validate parking entry and exit times and vehicle type, calculate parking duration and fee, and apply the maximum fee limit.

Procedure

57. Read entry hour, exit hour and vehicle type.
58. Check whether both hours are within 0–23; otherwise display invalid time.
59. Check whether the vehicle is car or bike; otherwise display invalid vehicle.
60. Calculate parking hours using modulo 24; if the result is zero, treat it as 24 hours.
61. Set the hourly rate to ₹30 for a car or ₹10 for a bike.
62. Calculate the fee as the smaller of hours multiplied by rate and ₹300.
63. If exit is earlier than entry, add ₹50.
64. Display parking hours and total parking fee; stop.

Flowchart



Program

```
entry = int(input("Enter entry hour (0-23): "))
exit = int(input("Enter exit hour (0-23): "))
vehicle = input("Vehicle (car/bike): ").lower()
if not (0 <= entry <= 23 and 0 <= exit <= 23):
```

```
    print("Invalid time")
elif vehicle not in ["car", "bike"]:
    print("Invalid vehicle")
else:
    hours = (exit - entry) % 24
    hours = hours if hours > 0 else 24
    rate = 30 if vehicle == "car" else 10
    fee = min(hours * rate, 300)
    if exit < entry:
        fee += 50
    print("Parking hours:", hours)
    print("Total parking fee: Rs.", fee)
```

Output

Sample input:
Enter entry hour (0-23): 9
Enter exit hour (0-23): 12
Vehicle (car/bike): car

Output:
Parking hours: 3
Total parking fee: Rs. 90

Experiment 10: Hospital Billing System

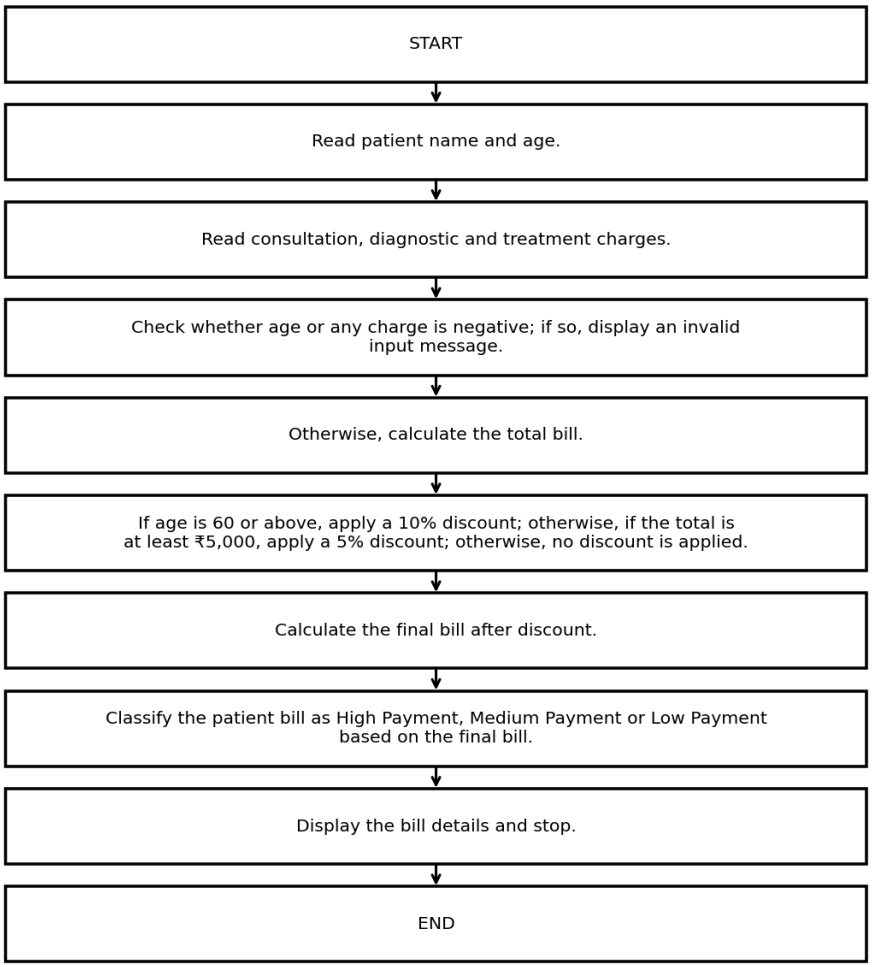
Aim

To prepare a hospital bill by validating patient details and charges, applying an eligible concession and classifying the final bill amount.

Procedure

65. Read patient name and age.
66. Read consultation, diagnostic and treatment charges.
67. Check whether age or any charge is negative; if so, display an invalid input message.
68. Otherwise, calculate the total bill.
69. If age is 60 or above, apply a 10% discount; otherwise, if the total is at least ₹5,000, apply a 5% discount; otherwise, no discount is applied.
70. Calculate the final bill after discount.
71. Classify the patient bill as High Payment, Medium Payment or Low Payment based on the final bill.
72. Display the bill details and stop.

Flowchart



Program

```
name = input("Enter patient name: ")
age = int(input("Enter patient age: "))
```

```
consultation = float(input("Enter consultation charge: "))
diagnostic = float(input("Enter diagnostic charge: "))
treatment = float(input("Enter treatment charge: "))

# Validate charges
if age < 0 or consultation < 0 or diagnostic < 0 or treatment < 0:
    print("Invalid input! Values cannot be negative.")

else:
    total = consultation + diagnostic + treatment

    # Apply concession
    if age >= 60:
        discount = total * 0.10
    elif total >= 5000:
        discount = total * 0.05
    else:
        discount = 0

    final_bill = total - discount

    # Classify patient
    if final_bill >= 10000:
        category = "High Payment"
    elif final_bill >= 5000:
        category = "Medium Payment"
    else:
        category = "Low Payment"

    # Display bill
    print("\n--- HOSPITAL BILL ---")
    print("Patient Name:", name)
    print("Total Bill: Rs.", total)
    print("Discount: Rs.", discount)
    print("Final Bill: Rs.", final_bill)
    print("Patient Category:", category)
```

Output

Sample input:
Patient name: Ravi
Patient age: 65
Consultation charge: 1000
Diagnostic charge: 3000
Treatment charge: 7000

Output:
--- HOSPITAL BILL ---
Patient Name: Ravi
Total Bill: Rs. 11000.0
Discount: Rs. 1100.0
Final Bill: Rs. 9900.0
Patient Category: Medium Payment